Federated Matching with Transformers

Krish Agarwal[1] **Marco Christiani**[1] **Jiang Meng Liao**[1]

[1]Electrical and Computer Engineering, Carnegie Mellon University

krisha2@andrew.cmu.edu marcochr@andrew.cmu.edu jiangmel@andrew.cmu.edu

Abstract

Naive FedAvg can face convergence issues when clients take a large amount of local steps per communication round and diverge from one another. A common source of divergence is heterogeneous data, but another major source of divergence can be weight permutation invariance of neural network architectures. This motivates finding approaches that account for model structure during weight aggregation to improve convergence. Some existing approaches show promising results, like FedMA, which performs channel-wise matching in convolutional neural networks to average the closest convolution kernel channels with each other between multiple client models. In this work, we find that similar permutation invariance exists in transformer architectures, specifically in attention blocks at a head-level granularity. To address this permutation invariance, we propose our method, *FedMAT*, to find a (partial) bipartite matching of attention heads between the global model and each client model in a federated learning setting to perform better client model aggregation and achieve faster convergence. Our work may be found online[1].

1 Introduction

Prior transformer-based FL methods such as FedTP (Li et al. 2024) face convergence challenges that arise from heterogenous client data. This causes clients to diverge from one another during local optimization steps, resulting in poor server aggregation.

Client divergence can be further exacerbated by permutation invariance in neural network architectures. Yurochkin et al. (2019) and Wang et al. (2020) have shown permutation invariance in FCNNs, CNNs, and LSTMs. FedMA (Wang et al. 2020) was proposed to address this problem for convolutional neural networks by exploiting activation channel-wise permutation invariance to average the closest convolution kernel channels with each other between multiple client models. FedMHA (Darzi et al. 2024) uses regularization in the local and global transformer updates in an attempt to improve alignment but does not take advantage of the permutation symmetry of multi-head attention. Most recently, Verma and Elbayad (2024) showed that explicitly matching and permuting attention heads before averaging can improve performance of the final merged model but did not consider the federated setting in their analysis.

We identify that multi-head attention is also permutation invariant in the ordering of its attention heads. As modern deep learning models are increasingly built using multi-head attention, we believe Federated Matching (Wang et al. 2020) can be adapted to the modern federated learning landscape. Our method *FedMAT* builds on the ideas of FedMA (Wang et al. 2020) and explores permutation invariance across attention heads in multi-head attention networks. We propose finding a (partial)

[1]Our code is made available at https://github.com/Marco-Christiani/fedmat

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

bipartite matching of attention heads with the global model with each client model in a federated learning setting to perform better client model aggregation and achieve faster convergence.

2 Federated Matching for Transformers

2.1 Permutation Invariance of Transformers

A transformer block is permutation invariant over the order of the attention heads. We define an H-headed transformer block to be

$$\begin{aligned} &Q_i = XW_i^Q, K_i = XW_i^K, V_i = XW_i^V \\ &A_i = \mathrm{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_k}}\right) V_i \\ &O = [A_1 ... \quad A_H] W^O = [A_1 ... \quad A_H] \begin{bmatrix} W_1^O \\ \vdots \\ W_H^O \end{bmatrix} = \sum_{i=1}^{H} A_i W_i^O \end{aligned} \tag{1}$$

where $X \in \mathbb{R}^{n \times d}$, $W_i^Q, W_i^K \in \mathbb{R}^{d \times d_k}$, $W_i^V \in \mathbb{R}^{d \times d_v}$, $W^O \in \mathbb{R}^{H d_v \times d}$, and $W_i^O \in \mathbb{R}^{d_v \times d}$ denotes the d_v rows of W^O from $(i-1)d_v + 1$ to $i d_v$. Here, n is the number of tokens, d is the size of the input embeddings, d_k is the size of the intermediate per-head embeddings, d_v is the per-head output dimensionality, and H is the number of attention heads. Typically, $d_v = d_k$ and $d = H d_v$.

Taking some permutation $\pi(i) : [H] \to [H]$ of the order of attention heads, if we define $A_i^{(\pi)} = A_{\pi(i)}$ and $W_i^{O,\pi} = W_{\pi(i)}^O$, then

$$\begin{aligned} O^{(\pi)} &= \sum_{i=1}^{H} A_i^{(\pi)} W_i^{O,\pi} = \sum_{i=1}^{H} A_{\pi(i)} W_{\pi(i)}^O \\ &= \sum_{i \in \{\pi(1), ..., \pi(H)\}} A_i W_i^O = \sum_{i=1}^{H} A_i W_i^O = O \end{aligned} \tag{2}$$

By invoking associativity of addition, we prove that a multi-head attention block is in invariant over permutation of the order of the attention heads, if the corresponding rows in W^O are likewise permuted.

2.2 FedMAT Algorithm

In Federated Matching for Transformers (FedMAT), for each client c per communication round t, we find some permutation $\pi^{(c,t)}(i) : [H] \to [H]$ operating on the following $2d(d_k + d_v)$-dimensional vectors

$$\left\{ \mathrm{concat}\left(\mathrm{flatten}\left(W_i^Q\right), \mathrm{flatten}\left(W_i^K\right), \mathrm{flatten}\left(W_i^V\right), \mathrm{flatten}\left(W_i^O\right) \right) \right\}_{i=1}^{H} \tag{3}$$

for every attention head in a multi-head attention block. In FedMA, a permutation applied at one layer has to be corrected using inverse permutations for subsequent layers, prompting FedMA to use a layer-wise training schedule so permutations/inverse permutations only need to be accounted for across client models for one layer at a time. In FedMAT, we apply permutation not only to the per-head W_i^Q, W_i^K, W_i^V weights, but we also apply the same permutation on the corresponding rows of the output projection W_i^O. This means the permutation is applied along the same dimension that is reduced when performing matrix multiplication of the attention output A with the output projection W^O. As such, the permutations applied at a layer are fully self-contained (applying a permutation at one layer requires no downstream correction for any other layers). This enables us to avoid any layer-wise training schedule. Instead we train every layer during every step, and we perform the matching independently per layer.

To perform matching (finding the correct permutation per client), we experiment with two methods, including FedMA's extended Hungarian matching algorithm (Kuhn 1955) and a custom greedy matcher, shown in Algorithm 1.

Algorithm 1 Greedy matching algorithm

procedure GreedyMatch(C, H, X, R, dist)
  ▷ C: number of clients
  ▷ H: number of attention heads
  ▷ $X = \left\{\left(x_{g,i}\right) \mid g \in [C], i \in [H]\right\}$: head-wise matching vectors for all clients
  ▷ $R = \left\{\left(r_j\right) \mid j \in [H]\right\}$: ordered reference head-wise matching vectors
  ▷ $\mathrm{dist}(\cdot, \cdot)$: a distance metric
  ▷ Output: $\left(\pi^{(1)}, ..., \pi^{(n)}\right)$ where $\pi^{(g)} \in [H]^H$ and $\pi_i^{(g)}$ is the index of the reference matched to $x_{g,i}$
  for $g = 1, ..., C$ **do**
    $U \leftarrow \{1, ..., H\}$ ▷ set of unused reference indices
    for $i = 1, ..., H$ **do**
      $j^* \leftarrow \mathrm{argmin}_{j \in U}\ \mathrm{dist}(x_{g,i}, r_j)$
      $\pi_i^{(g)} \leftarrow j^*$
      $U \leftarrow U \setminus \{j^*\}$
    end
  end
  return $\left(\pi^{(1)}, ..., \pi^{(C)}\right)$
end

FedMA (Wang et al. 2020) includes provisions in its matching algorithm for the creation of new convolutional kernel channels for CNNs (which would translate to new attention heads in our case), but we will not explore this in our report due to compute and time constraints.

3 Experiments

3.1 Setup

We chose to run our experiments on the CIFAR-10 dataset; while we initially planned to train our model on the ImageNet dataset, we found that the dataset was too challenging for our models to converge within our compute and time constraints.

We selected the AdamW optimizer as the best performing optimizer, with a learning rate of $1 \cdot 10^{-4}$ and weight decay of $1 \cdot 10^{-2}$.

We choose a batch size of 64, with 10 clients, using inverse class frequency weighted Focal Loss (Lin et al. 2018). We used several data augmentation techniques including random flips and rotations, gaussian noise, color jitter, and random erasure. We run each client for 500 local iterations before aggregating in a communication round, for 48 communication rounds.

We varied the homogeneity of class distributions between the clients as well, using a Dirichlet distribution with tunable concentration to sample a categorical distribution of clients for each class of image in our training set. We perform a strict partition of the full training set between clients.

3.2 Matcher comparison

Considering a highly homogenous distribution of training data between clients, we notice that the models perform identically, as shown in Figure 1. This is likely because the client models are all initialized using the same random noise, and in the first communication round, the client models do not diverge significantly relative to the magnitude of noise, causing both the greedy and Hungarian matchers to always select the identity permutation for all clients. This persists for all communication rounds.

We perform additional experiments to encourage more client drift. These include experiments that use separate random initialization per client (as shown in Figure 2) as well as a high initial learning rate with learning rate decay (as shown in Figure 3). In these cases, we see that more aggressive client drift does indeed lead to differences in the results (i.e. the greedy and Hungarian matchers do not select the identity permutation). In some cases, greedy or Hungarian matching can marginally exceed the performance of vanilla FedAvg, although any improvement is very minor.

We also include additional experiments under more aggressive data heterogeneity conditions in Section 5. However, we observe that using data heterogeneity does not significantly impact the relative comparison between matching strategies.

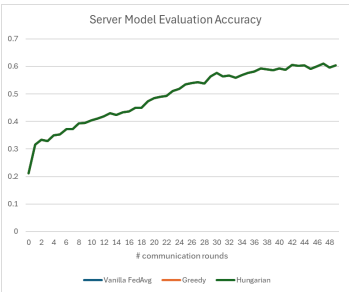

Figure 1: Matcher comparison with homogenous data ($\alpha = 100$)

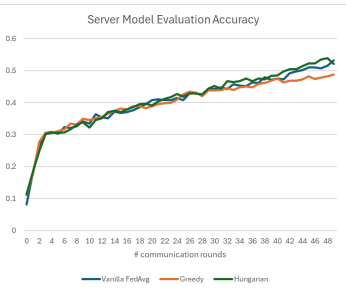

Figure 2: Matcher comparison with independently initialized clients and homogenous data ($\alpha = 100$)

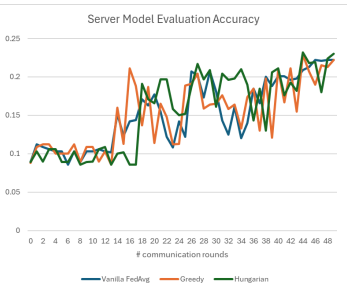

Figure 3: Matcher comparison with high learning rate ($\eta = 0.1, \alpha = 100$)

4 Conclusion and Future Works

We find that there is no significant difference in performance between FedAvg and matched averaging methods on transformers. We believe that there are a few reasons that lead to this negative result.

First, we believe that the strongly initialized, high-parameter count attention heads are less susceptible to the same kind of cluster-shifting behavior that the authors of FedMA found for convolution kernel channels. We believe this is because attention heads typically are much larger than a convolution channel in terms of parameter count, which makes it much less likely that any sort of matching would lead to a reassignment of attention head order as a client would need to shift many more parameters to cause a difference in clustering.

Second, we believe that our holistic training method, as compared to FedMA's layer-by-layer training method, leads to the model gradients to be dispersed throughout all layers. This means that the attention heads in each layer will have less drift between clients and further reduce the effectiveness of matched averaging compared to naive averaging.

Future studies could explore these hypotheses by measuring matching behavior and the drifting of individual attention heads. Reducing trainable parameter count could also be a viable way to bring out the benefit of matched averaging, using methods such as LoRA and reducing the numbers of attention heads per block.

5 Appendix

Here, we present some additional experiments that we conducted to better justify our earlier claims.

In Figure 4, we present an ablation of using various optimizers with vanilla FedAvg. As shown, AdamW achieves the highest performance (under homogenous client distributions), hence we select AdamW as the optimizer for all other experiments.

In Figure 5, we show that performance of the server model significantly drops under data heterogeneity, as would be expected. This is particularly because we experiment under a high number of local steps, so with significant client divergence due to data heterogeneity, we naturally get poorer convergence.

In Figure 6, we show that performance between matchers is identical under heterogenous datasets as well. Similarly, the relative performance of the matchers does not change significantly when using separate random initialization per client, as shown in Figure 7. We would have expected that more data heterogeneity leads to more client drift, which might make permutation invariance more of an issue, but this does not seem to be the case.

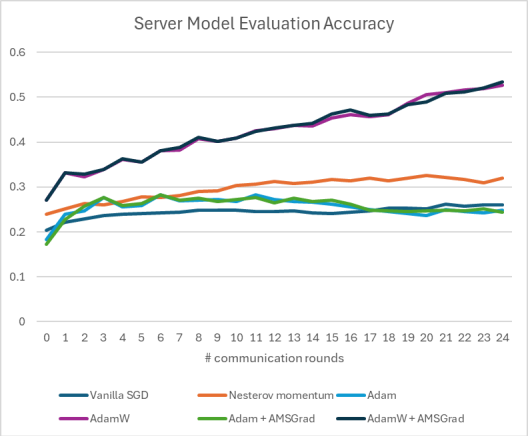

Figure 4: Comparison of various optimizers for vanilla FedAvg (homogenous, $\alpha = 100$)

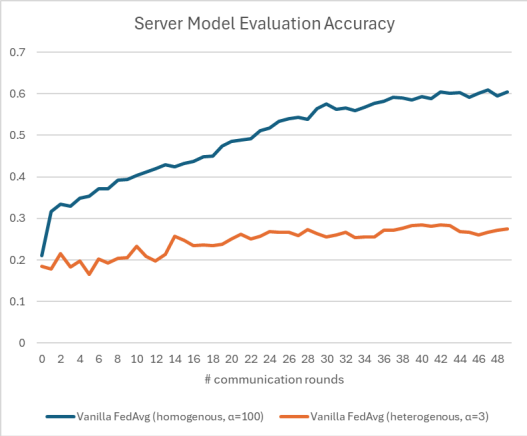

Figure 5: Dirichlet concentration comparison with FedAvg

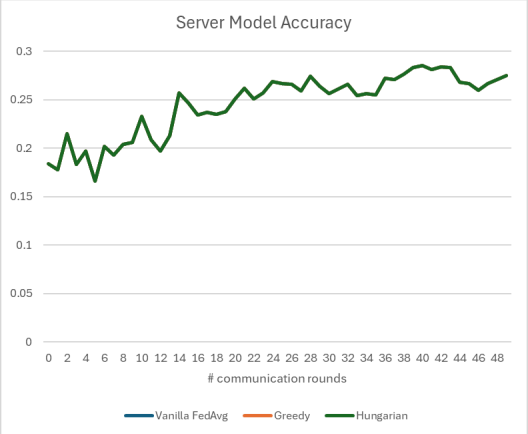

Figure 6: Matcher comparison with heterogenous data ($\alpha = 3$)

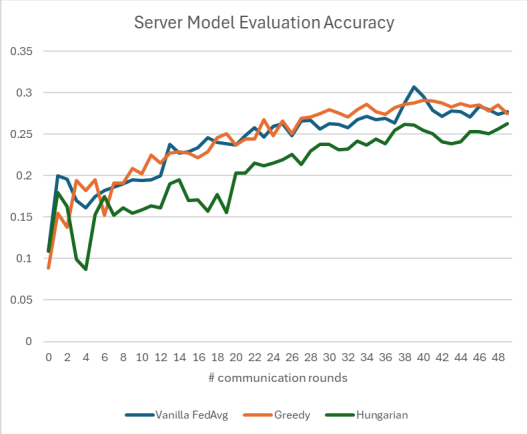

Figure 7: Matcher comparison with independently initialized clients (heterogenous, $\alpha = 3$)

Alexander, Jay, and Michael C Mozer. 1994. "Template-Based Algorithms for Connectionist Rule Extraction." *Advances in Neural Information Processing Systems* 7.

Bower, James M, and David Beeman. 2012. *The Book of GENESIS: Exploring Realistic Neural Models with the General Neural Simulation System*. Springer Science & Business Media.

Darzi, Erfan, Yiqing Shen, Yangming Ou, Nanna M. Sijtsema, and P.M.A van Ooijen. 2024. "Tackling Heterogeneity in Medical Federated Learning via Aligning Vision Transformers." *Artificial Intelligence in Medicine* 155 : 102936. https://doi.org/https://doi.org/10.1016/j.artmed.2024.102936.

Deng, Jia, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. "Imagenet: A Large-Scale Hierarchical Image Database." In "2009 IEEE Conference on Computer Vision and Pattern Recognition." Special issue, *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 248–55.

Hasselmo, Michael E, Eric Schnell, and Edi Barkai. 1995. "Dynamics of Learning and Recall at Excitatory Recurrent Synapses and Cholinergic Modulation in Rat Hippocampal Region Ca3." *Journal of Neuroscience* 15 (7): 5249–62.

Krizhevsky, Alex. 2009. *Learning Multiple Layers of Features from Tiny Images*. Technical report.

Kuhn, H. W. 1955. "The Hungarian Method for the Assignment Problem." *Naval Research Logistics Quarterly* 2 (1–2): 83–97. https://doi.org/https://doi.org/10.1002/nav.3800020109.

Li, Hongxia, Zhongyi Cai, Jingya Wang, et al. 2024. "Fedtp: Federated Learning by Transformer Personalization." *IEEE Transactions on Neural Networks and Learning Systems* 35 (10): 13426–40. https://doi.org/10.1109/TNNLS.2023.3269062.

Lin, Tsung-Yi, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2018. "Focal Loss for Dense Object Detection.". https://arxiv.org/abs/1708.02002.

McMahan, Brendan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. 2017. "Communication-Efficient Learning of Deep Networks from Decentralized Data." In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, edited by Aarti Singh and Jerry Zhu, vol. 54 of *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*. Proceedings of Machine Learning Research. PMLR. https://proceedings.mlr.press/v54/mcmahan17a.html.

Verma, Neha, and Maha Elbayad. 2024. "Merging Text Transformer Models from Different Initializations." *Transactions on Machine Learning Research*,. https://openreview.net/forum?id=nWnYSLncXa.

Wang, Hongyi, Mikhail Yurochkin, Yuekai Sun, Dimitris Papailiopoulos, and Yasaman Khazaeni. 2020. "Federated Learning with Matched Averaging." In "International Conference on Learning Representations." Special issue, *International Conference on Learning Representations*,. https://openreview.net/forum?id=BkluqlSFDS.

Wu, Bichen, Chenfeng Xu, Xiaoliang Dai, et al. 2020. "Visual Transformers: Token-Based Image Representation and Processing for Computer Vision.".

Yurochkin, Mikhail, Zhiwei Fan, Aritra Guha, Paraschos Koutris, and XuanLong Nguyen. 2019. "Scalable Inference of Topic Evolution via Models for Latent Geometric Structures." In *Advances in Neural Information Processing Systems*, edited by H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, vol. 32 of *Advances in Neural Information Processing Systems*. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2019/file/31c0c178a9fc26ffecffd8670e6d746d-Paper.pdf.